

LEFT JOIN OR Anti-Pattern

#sql-server #performance #join-strategy #refactoring

The Pattern

You have a row that can qualify through two different join paths. Path A goes through tables X and Y. Path B goes through tables M and N. The developer writes LEFT JOINS to all four tables and uses OR in the WHERE clause to accept either path:

```
Select i.Id, i.ContentCode
From Images i
Left Join RxAssoc IR On i.Id = IR.ImgId
Left Join OrderHistory OH On IR.OrderId = OH.OrderId
Left Join CanisterAssoc IC On i.Id = IC.ImgId
Left Join Canister C On IC.CanisterSn = C.CanisterSn
Where (i.Id = IR.ImgId Or i.Id = IC.ImgId)           -- at least one path matched
      And (OH.OrderStatus = 'Shipped' Or C.Status = 'Verified') -- path-specific filter
```

Why This Hurts the Optimizer

Three things go wrong at once.

1. LEFT JOINS Prevent Early Elimination

With INNER JOINS, the optimizer can drop non-matching rows at the join step. With LEFT JOINS, every row from the left table survives the join. The engine must produce the full result before the WHERE clause filters it down.

2. OR Across Join Paths Blocks Index Strategy

The optimizer needs to satisfy Condition_A OR Condition_B where A and B involve different tables. It can't seek on table X's index AND table M's index simultaneously for an OR. Typical outcomes:

- An Index Union or Concatenation plan, where the optimizer seeks each path separately and unions the results. This is the "lucky" outcome but often has high startup cost.
- A full scan plus hash match, where the optimizer gives up on seeks entirely and scans the base table. Common with complex ORs.
- Nested loops with a per-row filter, the worst case.

3. Cardinality Estimation Falls Apart

The optimizer struggles to estimate how many rows satisfy A OR B when A and B have different selectivities across different tables. It often falls back to a fixed guess (30% or the "magic 10%"), leading to wrong join strategies and undersized memory grants.

Fix #1: UNION Branches

The textbook refactor splits the query into two branches, one per join path, and combines them with UNION:

```
-- Branch 1: Shipped Rx Images
Select i.Id, i.ContentCode
From Images i
Join RxAssoc IR On i.Id = IR.ImgId
```

```

Join OrderHistory OH On IR.OrderId = OH.OrderId
Where i.ImageData Is Not Null
      And OH.OrderStatus = 'Shipped'

Union

-- Branch 2: Verified Canister Images
Select i.Id, i.ContentCode
From Images i
Join CanisterAssoc IC On i.Id = IC.ImgId
Join Canister C On IC.CanisterSn = C.CanisterSn
Where i.ImageData Is Not Null
      And C.Status = 'Verified'

```

Why UNION Helps

1. Each branch uses INNER JOINS, so the optimizer can push predicates into the join tree and use index seeks.
2. Each branch gets its own sub-plan with accurate cardinality estimates.
3. The optimizer never has to reason about an OR that spans different join trees.
4. UNION (not UNION ALL) handles deduplication automatically. Use UNION ALL only when you can guarantee a row cannot match both paths.

When UNION Doesn't Help

UNION isn't always the right answer. The pattern fails when the dominant cost is a scan of the shared base table that the original plan was already amortizing across both join paths. Splitting into two branches forces the engine to read that base table twice, once per branch, and the doubled scan cost can outweigh the index-seek benefit. For `Isp_ImgGetListOfTopXImagesToMove`, the UNION refactor measured 12-20% worse on logical reads than the LEFT JOIN + OR original because of exactly this effect on the 10.5M row `vImgImage` table. See Fix #2 below.

Fix #2: Materialize-Once + EXISTS

When the base table is large and qualifying rows are sparse, the better refactor is to scan the base table once into a temp table, then probe each path with EXISTS:

```

-- Step 1: Pull qualifying candidates into a temp table (one base-table pass)
Select i.Id, i.ContentCode
Into #Candidates
From Images i With (NoLock)
Where i.ImageData Is Not Null
      And (i.IsMovedToFile Is Null Or i.IsMovedToFile <> 1)
      And i.ArchivedDtTm Is Null

Create Clustered Index IX_C On #Candidates (Id)

-- Step 2: Apply path-specific filters via EXISTS
Select Top (@MaxImages) Id, ContentCode
From #Candidates c
Where Exists (
    Select 1 From RxAssoc IR With (NoLock)
    Inner Join OrderHistory OH With (NoLock) On IR.OrderId = OH.OrderId

```

```

        Where IR.ImgId = c.Id
        And OH.OrderStatus = 'Shipped')
Or Exists (
    Select 1 From CanisterAssoc IC With (NoLock)
    Inner Join Canister C With (NoLock) On IC.CanisterSn = C.CanisterSn
    Where IC.ImgId = c.Id
        And C.Status = 'Verified')
Order By Id Asc

```

Why This Pattern Works

1. The base table is scanned exactly once, in step 1. No doubled scan cost.
2. The temp table has real statistics, so the optimizer plans step 2 with accurate row counts.
3. Each EXISTS subquery is a separate sub-plan with its own join order and index choices. The optimizer can pick a good index per path without being constrained by the shape of the other path.
4. TOP combined with the Left Semi Join produced by EXISTS lets the optimizer apply rowgoal. As soon as enough rows have qualified, scanning of the temp table stops.
5. Path-specific WHERE filters live inside the EXISTS clauses, so the optimizer can choose path-appropriate indexes (for example, an index on OrderHistory.OrderStatus instead of being forced into PK_OrderHistory by an outer join structure).

When to Reach for This Pattern

- The shared base table is large enough that scanning it twice is meaningfully expensive.
- The path-specific filters are selective enough that EXISTS will short-circuit quickly.
- TOP is in play, so rowgoal can produce a real win.
- An OR across paths is genuinely the right semantics (at least one path matches).

Recognizing the Pattern in the Wild

Red flags in existing code:

1. Multiple LEFT JOINS where the WHERE clause references columns from those LEFT-joined tables. This converts the LEFT to an effective INNER, but the optimizer doesn't always recognize it cleanly.
2. OR conditions in WHERE that span different table aliases.
3. "At least one must match" logic, like (A.col = X Or B.col = Y).
4. FORCESEEK hints on the joined tables. Someone tried to fix the resulting scans with a hint instead of fixing the structure.

In SET STATISTICS IO, you'll see the base table racking up reads far in excess of what a TOP N query should need, and multiple joined tables showing high reads even though only one path is active per row.

Choosing Between the Two Fixes

Symptom	Pattern to Try
Each join path probes a small driver, base table is small	Fix #1 (UNION)
Base table is large, qualifying rows are sparse	Fix #2 (Temp Table + EXISTS)
TOP N with rowgoal is in play	Fix #2 generally wins
You can guarantee mutual exclusion across paths	Fix #1 with UNION ALL
Path-specific filters need different indexes than the outer query	Fix #2

If you're not sure, run both against the same data and compare logical reads, CPU, and elapsed time. Don't trust one-shot results, particularly cold-cache results.

Real-World Example

The `lsp_imgGetListOfTopXImagesToMove` refactor walked through both fixes and arrived at Fix #2.

The proc averages 1.14M logical reads per execution in production over a 30-day window. The base query LEFT JOINed four tables (`ImgRxImgAssoc`, `OeOrderHistory`, `ImgCanImgAssoc`, `CanCanister`) and used OR across two path-specific filters. FORCESEEK hints had been added to the association tables in an earlier maintenance pass.

We first tried Fix #1 (UNION of two INNER JOIN branches). On apples-to-apples data, the UNION variant produced 1.03M logical reads against the original's 919K. Reads went up, not down. Drilling into per-table reads showed why: `ImgImage` was being read twice, once per UNION branch, and the LOB read count on the `ImageData` column doubled too. The UNION shape converted a single shared scan into two separate scans of a 10.5M row table.

We then tried Fix #2: pull qualifying images into `#qualifyingImages`, then probe each path with EXISTS. The plan diff showed two structural wins. First, the optimizer switched from `PK_OeOrderHistory` to a `ByOrderStatus` index, which gave the `Shipped` filter a selective seek instead of a residual filter against PK seeks. Second, TOP combined with the Left Semi Join from EXISTS produced an effective rowgoal: only 9,069 of the 66,813 candidate temp-table rows were probed before TOP was satisfied. Final result on the same data: 214K reads (down 75% from the original), 0 LOB reads, 1.18s CPU (down 57%).

The cold-cache elapsed time came in higher than the original because the `ByOrderStatus` index pages weren't in the buffer pool. Warm-cache runs brought elapsed below the original baseline.

Related Concepts

- **Index Key Columns vs Included Columns**: picking the right index for the path-specific filter is half the battle in Fix #2.
- **UNION ALL Views**: another UNION-related optimization pattern.
- **Correlated Subqueries to CTEs**: another per-row access pattern that should be set-based.
- **FORCESEEK Hints**: often applied as a band-aid for this anti-pattern.
- **Non-SARGable Predicates**: another pattern that blocks index seeks.
- **Table Variables vs Temp Tables**: Fix #2 depends on the temp table having real statistics.